AI Control Plane — A Governed Agent Network in Production

CruxOCM · Feb 2025 – Present · Architect and owner · GKE, Kuma, LangGraph, MCP, Langfuse

Adding an agent to the network is a registration, not a project.

THE PROBLEM

Agents were multiplying faster than the governance around them: bespoke auth per agent, hand-wired tool access, no cost attribution, and no way to answer the two questions an enterprise always asks — *what is this thing allowed to touch, and how do we know its output was right?* Writing the agent was never the bottleneck. Making each new one safe, observable, composable, and reproducible was.

THE DECOMPOSITION — AGENTHOOD AS A SCHEMA

I made agenthood a declared contract rather than an implementation detail. To join the network, an agent declares: **identity and auth** (who it is, who may invoke it) · **route** (its path under `/agents/*` or `/mcp/*`) · **tool surface** (which MCP backends it may call) · **egress allowlist** (every destination it may reach) · **model tier** · **tenant budget**, enforced at the gateway · **approval requirement**, whether consequential actions pass a human gate · **evaluation set**, how its output quality is measured over time. That declaration is the ontology. It is also what makes agent-generated agents tractable: a generating agent emits a contract and a tool binding, and the platform enforces the rest.

THE ARCHITECTURE

Four layers on GKE with a Kuma service mesh — mTLS in-mesh, the cluster as the trust boundary:

- **Agent Gateway** — one URL fronting `/agents/` *and* `/mcp/`: authn, authz, guardrails, agent + MCP registration, path routing, model routing, per-tenant budgets, audit.
- **Control** — a LangGraph supervisor orchestrator routing intent to specialist agents, a shared memory store, and a human-in-the-loop approval gate; operator UI for approvals, config, and policy.
- **Execute** — specialist agents and MCP backends (ephemeral infra, InfluxDB, Grafana, signal editor/composer, HMI, project config, CI/CD, TestBOT), each with an egress allowlist and short-lived creds.
- **Observe** — Langfuse in-cluster: prompts, traces, tokens, cost, evaluations, datasets.

OUTCOMES

Onboarding a new agent became a **registration** rather than an integration project — the growth mechanism the network needed. Models route by workload tier, per-tenant budgets are enforced at the gateway instead of discovered on the invoice, every turn is traced with tokens, cost, and evaluations, and autonomy is a per-agent policy rather than a default. It is the substrate for **IAH**, a commercial multi-tenant B2B SaaS AI platform I architected 0→1 and sold to external customers across Azure and GCP, on a platform held at **99.99% uptime** under **ISO 27001 / SOC 2**.

WHY IT MAPS TO THIS MISSION

Agentic layering needs a floor to stand on. Agents cannot reliably create, orchestrate, and evolve other agents until agenthood is machine-readable, registration is programmatic, and every produced action is gated, budgeted, and traced. This is that floor, running in a safety-critical enterprise context — built on one principle: *the model proposes, the system disposes.*